# Frame Aligner - Bugs Found

I found three bugs in `Frame_aligner.sv`. All three show up at the DUT outputs.

I began with a list of how I expect the DUT to behave and what its goal is. Then I created a verification plan. I wrote down all of the corner cases and the behavior I want to verify works correctly. I moved on to create the environment and ran a few tests to confirm the basic assumptions for the rest of the tests. Then I created sequences and directed tests on purpose to check all the corner cases from the verification plan.

### --- The bugs found ---

### 1. False header inside the payload while aligned ( `frame_detect = 1` )

Once the block is aligned, it should only look for a header at the frame boundary. After a frame with a bad header, while frame_detect is still 1, the DUT searches for a header on every byte instead of keeping the 12 byte grid. So if a payload happens to contain `AA AF` or `55 BA`, the aligner jumps to it and the 12-byte frame grid shifts to the wrong place.

I caught it with a directed test, `frame_false_lock_test`. The test sends 4 good frames, then 10 frames with a bad header and a header value planted inside the payload, each followed by 6 good frames. The planted position is chosen cyclically from 0 to 9, so every payload position is used once. Positions 0–8 get an `AA AF` pair, and position 9 gets a single `AA` right before the next frame's header. The chosen position is printed in the log, so each mismatch can be matched to its position. The same result appears with the other header ( `55 BA` ).

From the log (planted pair at position 6):

```
[SEQ] planted location = 6 (AA AF pair)
UVM_ERROR scoreboard.sv(73) @ 635000: [SCB_POS_MISMATCH] fr_byte_position mismatch @cycle 60!
Expected: 0, Got: 1 | frame_pos=10 rx_data=0x44
UVM_ERROR scoreboard.sv(73) @ 645000: [SCB_POS_MISMATCH] fr_byte_position mismatch @cycle 61!
Expected: 0, Got: 2 | frame_pos=11 rx_data=0x4a
UVM_ERROR scoreboard.sv(73) @ 655000: [SCB_POS_MISMATCH] fr_byte_position mismatch @cycle 62!
Expected: 0, Got: 3 | frame_pos=0 rx_data=0xaa
```

The DUT re-aligns on the planted pair and keeps counting from it (1, 2, 3…), while the real frames keep arriving on the original grid.

I ran two control tests ( `frame_payload_hdr_ok_test` ). Without the header bytes in the payload there is no error. With the same bytes sent while the block is not yet aligned, there is also no error, which is the correct behavior. So the bug only appears when the block is already aligned.

This bug is especially problematic because `frame_detect` stays 1 while the byte positions are wrong, so a module that relies on `frame_detect` gets no indication that something went wrong.

The random test (`frame_random_test`) shows another effect of this bug. This test also sends frames that are not 12 bytes long (variable-length payload). The reference model locked and unlocked 5 times, but the DUT locked once and never unlocked. Since it keeps locking onto fake headers inside the data, it never gets 48 bytes in a row without a legal frame, so `frame_detect` never drops. I treat this only as additional evidence and not as a separate bug, because it is caused by the same bug.

---

## 2. A valid LSB in place of the MSB is not re-checked (LSB, LSB, MSB)

When a header LSB is followed by a byte that is not the MSB, the DUT drops that byte, even if it is itself a valid LSB. For example, `AA AA AF` followed by 9 payload bytes has a legal `AFAA` header every 12 bytes, but the DUT never locks on it. The same happens when the two LSBs come from different header patterns (`AA 55 BA`, `55 AA AF`).

I caught it with a directed test, `frame_lsb_lsb_msb_test`. The test sends `AA AA AF` frames in three situations: while the block is aligned, while it is not aligned, and while it is in the middle of acquiring alignment. When the block is not aligned, the reference model locks on these frames, but the DUT's `frame_detect` stays 0.

- As a control, `frame_lsb_lsb_lsb_msb_test` sends `AA AA AA AF`, and here the DUT does lock. With three LSBs in a row the third one is picked up and the fourth byte completes the header. So the DUT misses only the case where the second byte should be checked again.
- `frame_cross_retest_test` sends `AA 55 BA` and `55 AA AF`, and fails the same way. Here the second byte belongs to the other header pattern, so the DUT isn't just staying on the pattern it started with. The bug happens with both patterns.

The state diagram in the spec also does not show this case. It should have a transition that checks the byte again as a new LSB when it is a valid LSB.

---

## 3. `fr_byte_position` shows 1 while the block is searching (LSB followed by an invalid byte)

When a header LSB is followed by a byte that is neither the MSB nor another LSB, the block should go back to searching with `fr_byte_position` at 0. Instead, the output shows 1 for one clock cycle and only then goes back to 0.

Waveform 2 in the spec shows the position staying at 0 here, but the state diagram does not show a position reset on this transition. This is a conflict inside the spec (waveform 2 vs. the state diagram). I followed waveform 2.

I caught it with a directed test, `frame_cross_hdr_test`. The test sends six frames with a mixed header (`AA BA` or `55 AF`), then four clean frames. This happens every time a valid LSB is followed by a byte that is not the MSB, so it appears in many tests, but in this test it is the only error.

From the log:

```
cycles checked       : 139
frame_detect errors  : 0
byte position errors : 6
lock events          : 1
unlock events        : 0
```

There is one position error per bad frame and no `frame_detect` errors at all, so the only wrong output is the position. After the bad frames, the four clean frames lock normally and the errors stop.

---

Bugs 2 and 3 start from the same event: a valid LSB followed by a byte that is not the MSB. They differ in what that byte is. If it is `AA` or `55`, it should be checked again as a new LSB (bug 2). If it is any other byte, the block should go back to searching with position 0 (bug 3).

I found both bugs the same way. My reference model was built from the state diagram, so at first it behaved the same as the DUT and reported nothing. Based on the spec, I expected a different behavior (lock on `AA AA AF`, position 0 while searching), so I checked the waveforms and saw that the DUT does not behave that way. After that I updated the model, so it now reports both bugs.